

TECHNICAL DESIGN DOCUMENT

Add Currency Field to Payment Requests and Enforce Proper Bank Transfer Validation

Generated: 2026-10-06T11:17:46.155727+00:00

#####

2. ISSUE SUMMARY

=====

The payment system lacks currency specification in requests and bank transfers allow invalid transactions

3. REQUIREMENTS

=====

Functional Requirements:

- Add a currency field to the PaymentRequest class.
- Validate the currency code for supported values before processing payments.
- Prevent bank transfers from withdrawing more than an account's balance.
- Reject bank transfers with zero or negative amounts.

Non-Functional Requirements:

Not specified.

4. CURRENT SYSTEM / ARCHITECTURE

=====

Currently, the orders service processes payments without specifying a currency. The bank transfer service

Diagram:

[orders]

--> [shipping]

--> [user]

[shipping]

--> [orders]

--> [user]

[user]

(no further dependencies)

[cart-service]

(no further dependencies)

[bank-transfer-service]

(no further dependencies)

5. CURRENT FLOW

=====

1. Process Payment Request

A payment request is created without specifying a currency.

2. Transfer Funds Between Accounts

TransferService allows transfers that can overdraw an account or use non-positive amounts.

Flow:

Process Payment Request

|

v

Transfer Funds Between Accounts

6. ROOT CAUSE / PROBLEM ANALYSIS

=====

The PaymentRequest class currently lacks a currency field, which prevents specifying or validating the cu

7. PROPOSED SOLUTION

=====

Introduce a currency field in PaymentRequest and add logic to validate its value against supported curren

8. PROPOSED ARCHITECTURE

=====

Enforced validation in both the payment and bank transfer processes enhance reliability and correctness.

Diagram:

[orders]

--> [shipping]

--> [user]

[shipping]

--> [orders]

--> [user]

[user]

(no further dependencies)

[cart-service]

(no further dependencies)

[bank-transfer-service]

(no further dependencies)

9. PROPOSED FLOW

=====

1. Validate Payment Currency

Check if the provided currency in a payment request is supported.

2. Check Account Balance Before Withdrawal

Ensure the account has sufficient balance for withdrawal during transfer.

3. Reject Non-positive Transfer Amounts

Verify that transfer amounts are strictly positive.

Flow:

Validate Payment Currency

|

v

Check Account Balance Before Withdrawal

|
v

Reject Non-positive Transfer Amounts

10. SEQUENCE / INTERACTION FLOW

=====

```
client --- Submit payment request with currency --> orders
orders <-- Validate currency and amount --- paymentAuth
client --- Request fund transfer --> bank-transfer-service
bank-transfer-service <-- Validate account and transfer amount --- bank-transfer-service
```

11. AFFECTED SERVICES

=====

Service: orders

Complexity: medium

Responsibility change: The Orders service extends its responsibilities to handle and validate the currency

Reason: Integrating currency into payment processing enhances the capability to handle diverse transactions

API changes:

- Add currency field to payment requests.

Service: bank-transfer-service

Complexity: medium

Responsibility change: The Bank Transfer Service is now responsible for ensuring the legitimacy of transfers

Reason: By adding validation, we prevent misuse of the transfer service that could otherwise lead to income loss

12. AFFECTED FILES, CLASSES & FUNCTIONS

=====

[orders]

Symbol: PaymentRequest

File: src/main/java/works/weave/socks/orders/resources/PaymentRequest.java

Change type: modify

Current behavior: Holds address, card, customer, and amount information only.

Change: Add a new field 'currency' to PaymentRequest and modify the constructor and field-access methods.

Reason: By including a currency field in the payment request, we can support multi-currency transactions

Implementation: Introduce a String field 'currency'. Update constructor to accept this field. Modify toString

Pseudocode (grounded in real source):

```
class PaymentRequest {
    private String currency;
    // Existing fields...

    public PaymentRequest(Address address, Card card, Customer customer, float amount, String currency) {
        // Existing assignments
        this.currency = currency;
    }

    public String getCurrency() {
        return currency;
    }

    public void setCurrency(String currency) {
        this.currency = currency;
    }
}
```

```
}  
}
```

Acceptance criteria:

- PaymentRequest class includes currency field.
- Currency can be set and retrieved as expected.
- Existing functionality remains unaffected except where modified.

[orders]

Symbol: PaymentResponse

File: src/main/java/works/weave/socks/orders/resources/PaymentResponse.java

Change type: modify

Current behavior: Handles payment authorization response.

Change: Implement currency validation logic.

Reason: Validating currency ensures that the payment is processed correctly and with a currency that the

Implementation: Add logic to check if the currency is among the supported types before proceeding with pa

Pseudocode (grounded in real source):

```
if (!isCurrencySupported(currency)) {  
    throw new UnsupportedOperationException();  
}  
// proceed if valid
```

Acceptance criteria:

- Payments with unsupported currencies are rejected.
- Payments with supported currencies proceed.
- Proper error message is returned for unsupported currencies.

[bank-transfer-service]

Symbol: Account.withdraw

File: src/main/java/com/bank/Account.java

Change type: modify

Current behavior: Subtracts amount from balance without checking sufficiency.

Change: Add check for balance sufficiency before withdrawing.

Reason: Preventing overdrafts maintains financial integrity and ensures accounts cannot be inadvertently

Implementation: Before reducing the balance, compare the withdrawal amount to current balance and throw a

Pseudocode (illustrative only, not verified code):

```
void withdraw(double amount) {  
    if (amount > balance) {  
        throw new IllegalStateException("Insufficient balance");  
    }  
    balance -= amount;  
}
```

Acceptance criteria:

- Insufficient balance results in an exception.
- Account balance remains unchanged if withdrawal is rejected.

[bank-transfer-service]

Symbol: TransferService.transfer

File: src/main/java/com/bank/TransferService.java

Change type: modify

Current behavior: Performs transfer without checking for positive amount.

Change: Add check for positive transfer amounts.

Reason: Ensuring transfer amounts are positive prevents errors in transaction processing and maintains lo

Implementation: Before proceeding with the transfer, ensure amount is strictly positive and throw an exce

Pseudocode (illustrative only, not verified code):

```
public Transaction transfer(Account from, Account to, double amount) {  
    if (amount <= 0) {  
        throw new IllegalArgumentException("Transfer amount must be positive");  
    }  
    // existing logic
```

}

Acceptance criteria:

- Zero or negative amounts result in an exception.
- Only positive transfer amounts are accepted.

13. API CHANGES

=====

orders:

- Add currency field to payment requests.

14. DATA MODEL / DATABASE CHANGES

=====

Affected data model: Unknown

15. INTER-SERVICE COMMUNICATION

=====

Real service-to-service dependencies (from the graph):

- orders -> shipping
- shipping -> orders
- shipping -> user
- orders -> user

16. ERROR HANDLING & EDGE CASES

=====

New exceptions thrown for unsupported currency, insufficient balance, and non-positive transfer amounts r

17. SECURITY CONSIDERATIONS

=====

Ensuring currency validation and transaction checks prevent potential errors but do not inherently introd

19. TESTING STRATEGY

=====

Category	Scenario	Expected Result
unit	Add and retrieve currency in PaymentRequest	Currency field is correctly set and r
unit	Validate currency against supported list	Payments with unsupported currencies
unit	Reject transfer if amount exceeds balance	Transfer is rejected with proper erro
unit	Reject zero or negative transfer amounts	Transfer is rejected with proper erro
integration	Integration test for payment processing with currency	Payments are processed correctly with
integration	Integration test for valid and invalid bank transfers	Valid transfers complete correctly, w

20. IMPLEMENTATION PLAN

=====

1. Add currency field to PaymentRequest class.

2. Implement currency validation logic in payment processing.
3. Update Account.withdraw method to check for sufficient balance.
4. Update TransferService.transfer method to check for positive amounts.
5. Write unit tests for new PaymentRequest currency handling.
6. Write integration tests for currency validation in payments.
7. Write unit tests for Account and TransferService validation logic.
8. Run integration tests to verify end-to-end process flow.

21. RISKS

=====

- Changes to payment API might require frontend/client updates if they don't supply currency now.
- Validation logic could reject valid transfers if implemented incorrectly, impacting user experience.

22. OPEN QUESTIONS

=====

- What are the supported currency codes for the payment service?
- Do any client applications need updating due to the new currency field in payment requests?

23. CHANGE SUMMARY

=====

Service	File	Symbol
orders	src/main/java/works/weave/socks/orders/resources/PaymentRequest.java	PaymentRequ
orders	src/main/java/works/weave/socks/orders/resources/PaymentResponse.java	PaymentResp
bank-transfer-service	src/main/java/com/bank/Account.java	Account.wit
bank-transfer-service	src/main/java/com/bank/TransferService.java	TransferSer

24. FILES TO CREATE / MODIFY / DELETE

=====

Files to Create:

None identified.

Files to Modify:

- src/main/java/com/bank/Account.java
- src/main/java/com/bank/TransferService.java
- src/main/java/works/weave/socks/orders/resources/PaymentRequest.java
- src/main/java/works/weave/socks/orders/resources/PaymentResponse.java

Files to Delete:

None identified.

25. OVERALL REASONING

=====

The proposed changes integrate currency specification into payment processing and enforce essential valid